VAPT · MANUAL TESTING

Web App Security Testing

A step-by-step manual penetration-testing guide
for a Shopify store (theme, content & custom apps)

Target class: e-commerce web application · Tooling: Burp Suite + browser
Method: manual, one test at a time · Scope: own test store only

Prepared as an interview-ready walkthrough

Contents

1. What VAPT is & how to talk about it
2. Scope & rules of engagement
3. Setup — tools, accounts, test mode
4. Methodology — the loop you repeat
5. Area 1 — Authentication (login, signup, password reset)
6. Area 2 — Access control (IDOR)
7. Area 3 — Input validation (XSS & injection)
8. Area 4 — Cart & checkout business logic
9. Area 5 — Exposed data (secrets & security headers)
10. Recording findings & rating severity
11. Writing the report
12. Appendix A — Practice labs
13. Appendix B — Payload & command cheat-sheet

1. What VAPT is & how to talk about it

VAPT = Vulnerability Assessment and Penetration Testing. Vulnerability assessment is the broad hunt for weaknesses; penetration testing is proving a weakness is really exploitable. In an interview, lead with the structure: *scope → recon → test each class manually → confirm → rate → report.*

This guide covers the application layer of an e-commerce site — specifically the parts **you control**: the theme, the content, and any custom apps. You test these against the OWASP-style classes below:

Area	Class	Core question
1	Authentication	Can I register weakly, learn who has an account, or abuse password reset?
2	Access control (IDOR)	Can I reach another user's data by changing an ID?
3	Input validation	Is my input cleaned, or does it execute (XSS) / inject?
4	Business logic	Can I tamper with price, quantity or discounts?
5	Information exposure	Are secrets in the page, and are security headers set?

> **Interview framing:** "I work manually, one variable at a time, so every finding is reproducible and I never cause load. I prove impact with the minimum evidence, rate it, and write it up with a fix."

2. Scope & rules of engagement

Agree these **before** touching anything. They protect you and the client.

- **Only the authorised host.** One test store, nothing else. If a redirect sends you off-domain, you stop at the boundary.
- **Manual only.** No automated scanners, no brute-force, no flooding. A human clicking pace is the rate.
- **Don't attack the platform.** On Shopify you test *your* customisations, not Shopify's own login, checkout rails or admin. If a bug looks like the platform's, **stop and report it**.
- **Test mode for payments.** Bogus Gateway / test cards only. Never a real card, never a real paid order.
- **Protect real data.** If a test would expose a real customer's data, prove it with one redacted record and stop — never harvest.

> **Stop-and-report triggers:** a platform bug, any route to another real customer's data, anything that could touch production, or accidental data change. Document, notify, don't keep probing.

3. Setup — tools, accounts, test mode

3.1 Tools

- **Browser** with a clean profile (one per identity when testing access control).
- **Burp Suite** (Community is enough for manual work):
  - Proxy the browser through Burp (the embedded browser avoids certificate setup).
  - **Target → Scope:** add the host, enable "show only in-scope".
  - You use: **Proxy** (observe), **Repeater** (edit & re-send one request), Decoder, Comparer.
  - You do **not** use: Active Scanner, high-volume Intruder/fuzzing.

3.2 Test accounts

Create at least **two** of your own customer accounts — you prove access-control bugs by logging in as User A and trying to reach User B's data. Use `+tag` email addressing so both land in an inbox you control (needed for reset-link testing). Record each account's numeric **customer ID** and **order IDs**.

Label	Email	Customer ID	Order IDs
User A (attacker)	you+a@example.com	____	____
User B (victim)	you+b@example.com	____	____

3.3 Baseline recon (read-only, do first)

Browse the store once with Burp recording passively to map *your* surfaces: home, collections, product pages, search, cart, account area, login/signup/reset, any custom-app pages and `/apps/*` routes. Note which pages are yours vs Shopify-native — your testing concentrates on yours.

4. Methodology — the loop you repeat

Every test, in every area, is the same five-step loop. Say this in the interview:

1. **Observe** the request/response in Burp.
2. **Form a hypothesis** ("the server trusts this value").
3. **Change one variable** in Repeater — never two at once.
4. **Compare** the result to the secure expectation (PASS vs FAIL).
5. **Record** a FAIL as a finding with evidence; move on.

> Discipline is the point: one variable at a time, a human pace, and evidence for everything. That's what separates a tester from a scanner.

5. Area 1 — Authentication

Surfaces: login, signup, password reset, session handling.

1.1 Weak password policy

Goal: register or change to a trivially weak password.

Steps: at signup / change-password, try `123456`, `password`, the email local-part, a 1-character password.

FAIL if any is accepted. **PASS** if rejected with a real policy (length + complexity, or a breached-password check).

1.2 Username / email enumeration

Goal: tell whether an email has an account.

Steps: compare responses for a *known-registered* email vs a *random* one at three places — login (wrong password), signup, and password reset. Use Burp **Comparer** for text, status, length *and* timing differences.

FAIL if the responses differ in a way that reveals existence (e.g. "no account" vs "wrong password"). **PASS** if uniform: "if an account exists, we've emailed you", same timing/length.

1.3 Password-reset link — expiry, reuse, predictability

Steps:

1. Trigger a reset for A, grab the link from the inbox.
2. **Reuse:** use it once, then try the same link again.
3. **Expiry:** request a fresh link, wait past the claimed TTL, try it.
4. **Invalidation:** request link #1, then link #2 — does #1 still work?
5. **Token quality:** is it long and random, or guessable (sequential, a timestamp, base64 of the user id)?

FAIL if reusable after use, non-expiring, old link still valid after re-issue, or token predictable / not bound to the account.

1.4 Session hygiene (quick checks)

- After logout, replay a pre-logout authenticated request in Repeater — does the old session still work? (FAIL if yes.)
- Inspect session-cookie flags: `Secure`, `HttpOnly`, `SameSite`.

Note: if the session is a core Shopify customer session, that's likely platform — report, don't keep probing (scope rule).

6. Area 2 — Access control (IDOR)

The highest-value area. Every test has the same shape: **User A tries to read or modify User B's object by changing an identifier**, while keeping A's own session.

2.1 Orders

Steps: A opens their own order (note the URL/param, e.g. `/account/orders/<id>` or an order token). In Repeater, swap the id/token for B's order while keeping A's session cookie. Try sequential ids (`1001 → 1002`), the raw numeric id, and the confirmation token.

FAIL if A sees B's order. **PASS** if 403 / 404 / redirect to login.

2.2 Account / profile

Same pattern on profile view/edit, saved addresses, stored details. Swap `customer_id` / account slug in the URL, body, or a hidden form field. **FAIL** on read *or edit* of another account.

2.3 Custom-app endpoints & app proxy (`/apps/*`)

Custom apps often roll their own authorization, so they're prime IDOR targets. For each route that takes an id:

- Remove the id — does it default to "all" / someone else?
- Increment/guess the id as A — do you get B's record?
- Call it **logged out** — does it still return data?
- Tamper with any HMAC/signature param — is it actually verified, or accepted when you change `id` but keep the old signature?

2.4 Forced browsing

As anon and as a normal customer, try to reach admin-ish/internal pages your theme/app exposes (custom-app admin routes, debug pages, draft/unpublished URLs seen in the sitemap). **FAIL** if gated content renders without the right role.

Reminder: Shopify's own `/admin` auth is out of scope — this is about *your* app/theme pages.

7. Area 3 — Input validation (XSS & injection)

Test everywhere input is reflected or stored: search, forms, reviews, account fields, newsletter signup, custom-app inputs, and URL params echoed into the page.

3.1 Reflected XSS

Inject one payload at a time and watch where it lands in the response:

```
"><script>alert(1)</script>
"><img src=x onerror=alert(1)>
'"><svg onload=alert(1)>
```

FAIL if it executes or breaks out of the HTML/attribute context unescaped. **PASS** if rendered as inert, encoded text.

3.2 Stored XSS

Submit the same payloads into anything persisted — **product reviews**, account name/address, custom-app fields, order notes — then view the page that renders it back. Note the victim (other customers? a staff admin viewing reviews?). Stored XSS is more severe than reflected.

3.3 Attribute / HTML injection

Where `<script>` is stripped, test partial breakouts: unmatched quotes, `onmouseover=`, `style=`, an `<a href>`. Unquoted attributes break out without needing `<>`.

3.4 Server-side template injection

In custom-app fields, try `{{7*7}}`, `${7*7}`, `<%= 7*7 %>`. If the response shows `49`, input reached a template engine — document, don't run destructive payloads.

3.5 Other injection (light, manual)

- **SQL/NoSQL:** one `'` or `")` in a custom-app filter param — a 500 / DB error is a signal. Note it; do *not* run sqlmap (rule: no scanners).
- **Open redirect:** params like `?return_url=`, `?redirect=` set to `https://evil.example` — does it redirect off-domain?

8. Area 4 — Cart & checkout business logic

Golden rule: the server must recompute every price, quantity and discount itself. Each flaw is the server trusting a number the client sent. *Find the number, change the number, watch the total.* Test mode only.

4.1 Price tampering

Steps: add an item; in Burp find a `price` / `amount` in the request or a hidden field; in Repeater set it to `0.01`; proceed toward checkout.

FAIL if the server honours your price. **PASS** if the price is re-derived server-side.

4.2 Negative / overflow quantity

Set quantity to `-1`, `0`, a huge number or a decimal. **FAIL** if a negative quantity yields a negative line total (credit) or breaks inventory/overflow logic.

4.3 Discount reuse & stacking

Apply a single-use code twice; stack two codes; tamper with a client-supplied discount amount. **FAIL** if a code works beyond its limits, stacks when it shouldn't, or the value is attacker-controlled.

4.4 Totals / rounding / final charge

Confirm the tampered total reaches **checkout** (test mode), not just the cart preview — that's the difference between a display bug and a real financial finding.

If a flaw only works because of Shopify's own checkout rails, stop and report it (scope rule).

9. Area 5 — Exposed data (secrets & headers)

5.1 Secrets in source / JavaScript

View source of key pages and read every custom `*.js`. Grep for:

```
api_key  apikey  secret  token  password  Bearer  Authorization
AKIA...(AWS)   sk_live_/pk_live_(Stripe)   shpat_/shpss_(Shopify tokens)
```

Check HTML comments, inline config objects (`window.__CONFIG__`), source maps (`.js.map`), and `/apps/<app>` JSON responses. **FAIL** on any live secret. A leaked Shopify Admin/private-app token is **critical** — it can control the store.

5.2 Security headers

`GET` the main pages and check for: `Content-Security-Policy`, `Strict-Transport-Security`, `X-Content-Type-Options: nosniff`, `X-Frame-Options` / CSP `frame-ancestors`, `Referrer-Policy`, plus cookie flags on *your* cookies.

```
curl -sSI https://<target>/            # headers on home
curl -sS  https://<target>/ | grep -Eio 'api[_-]?key|secret|token|bearer|shpat_[a-z0-9]+'
```

Many headers are set by Shopify's edge and aren't yours to change — flag those as informational/platform, not your defect.

5.3 Information disclosure

Trigger a 404/500 on your routes — stack traces, framework banners, internal paths? Request a handful of files by hand (`/robots.txt`, `/sitemap.xml`, source maps, `.git/`, backups) — **no wordlist fuzzing**.

10. Recording findings & rating severity

Open one record per confirmed finding. Keep the raw request/response and a screenshot.

Finding record template

Field	Content
ID / Title	F-003 — IDOR on order history
Area & severity	Access control · High
Ownership	Your app/theme | Shopify platform (report & stop)
Location	URL / endpoint / parameter
Steps to reproduce	Numbered, exact
Request / Response	Burp raw, relevant part
Impact	What an attacker gains
Remediation	The fix

Severity rubric (CVSS-lite)

Severity	Rule of thumb	Example
Critical	Full compromise / mass data exposure, little effort	Leaked Admin API token; IDOR exposing all orders; price-to-zero that completes an order
High	Account takeover, cross-customer data, clear financial loss	Stored XSS hitting staff; IDOR reading another customer's orders; reset-link reuse
Medium	Meaningful but constrained	Reflected XSS; email enumeration; weak password policy; open redirect
Low	Minor info leak / hardening gap	Missing `X-Content-Type-Options`; cookie missing `SameSite`
Info	Not exploitable alone / platform-owned	Shopify-set headers; version banners

11. Writing the report

1. **Executive summary** — plain-language bottom line, a severity count table, and the top 3 risks in one sentence each.
2. **Scope & methodology** — what was tested, how, and the rules observed.
3. **Findings** — one section each, ordered by severity, with reproduction, evidence, impact, and fix.
4. **Remediation summary** — a quick table of fixes by issue class.
5. **Platform / out-of-scope items** — anything referred to Shopify.
6. **Appendix** — accounts used (labels only), timeline, tooling, evidence index.

> **Interview close:** "I'd hand over a report a developer can act on without me in the room — each finding reproducible, rated, and paired with a concrete fix."

Appendix A — Practice labs

Four local, deliberately-vulnerable practice targets mirror each testable area, so you can rehearse the exact techniques safely and offline before (or during) an interview. Each runs with Python's standard library only.

Area	Lab	Run	Flaws to prove
1	login_lab.py	`python3 login_lab.py`	weak password, enumeration, reset reuse
2	idor_lab.py	`python3 idor_lab.py`	order/account IDOR, no-auth API, forced browsing
3	xss_lab.py	`python3 xss_lab.py`	reflected/stored XSS, SSTI, open redirect
4	cart_lab.py	`python3 cart_lab.py`	price tampering, negative qty, discount reuse

Mobile/browser HTML versions also exist for areas 1, 2 and 4 (open on a phone, no server). Each flaw is labelled `# FLAW x.y` in the source, and the fix is in the remediation table.

Worked example — cart price tampering (real lab output)

```
$ python3 cart_lab.py                    # http://127.0.0.1:8003
# add mug (real £10) but send price=0.01, qty=2:
GET /cart/add?product=mug&qty=2&price=0.01
# /cart shows:
Total: £0.02        <-- server trusted the client price  (FAIL)

# negative quantity:
GET /cart/add?product=shirt&qty=-3
Subtotal: -£74.98   <-- negative line = credit           (FAIL)

# reuse a single-use discount three times:
GET /discount?code=SAVE10   (x3)
Discounts (3)       <-- stacked, no one-per-order rule   (FAIL)
```

Appendix B — Payload & command cheat-sheet

XSS

```
"><script>alert(1)</script>      "><img src=x onerror=alert(1)>
'"><svg onload=alert(1)>         javascript:alert(1)
{{7*7}}   ${7*7}   <%= 7*7 %>      (template injection -> 49)
```

IDOR

```
Keep YOUR session cookie, change only the id:
/account/orders/1001  ->  /account/orders/1002
/account?id=1001      ->  /account?id=1002
/api/<app>/order?id=   (call it logged out)
```

Cart tampering

```
price=0.01     qty=-1 / 0 / 999999     discount applied x3 / stacked
Then confirm the total at /checkout in TEST MODE.
```

Recon / headers / secrets

```
curl -sSI https://<target>/
curl -sS  https://<target>/ | grep -Eio 'api[_-]?key|secret|token|bearer|shpat_[a-z0-
9]+|sk_live_[a-z0-9]+|AKIA[0-9A-Z]{16}'
GET /robots.txt   /sitemap.xml   *.js.map
```

Burp tools used

```
Proxy    – observe requests/responses
Repeater – edit and re-send ONE request (Ctrl-R to send there)
Comparer – diff two responses (enumeration / timing)
Decoder  – encode/decode payloads
Scope    – keep everything in-bounds; NO Active Scan, NO Intruder fuzzing
```

> **Always remember:** authorised host only · manual, one variable at a time · test mode for payments · don't attack the platform — if a bug looks like Shopify's own, stop and report it.